# 深度学习与自然语言

2026年4月

# 本节课内容

- LoRA: Low-Rank Adaptation of Large Language Models
- MoE, Mixture-of-Experts

# 背景：全量微调

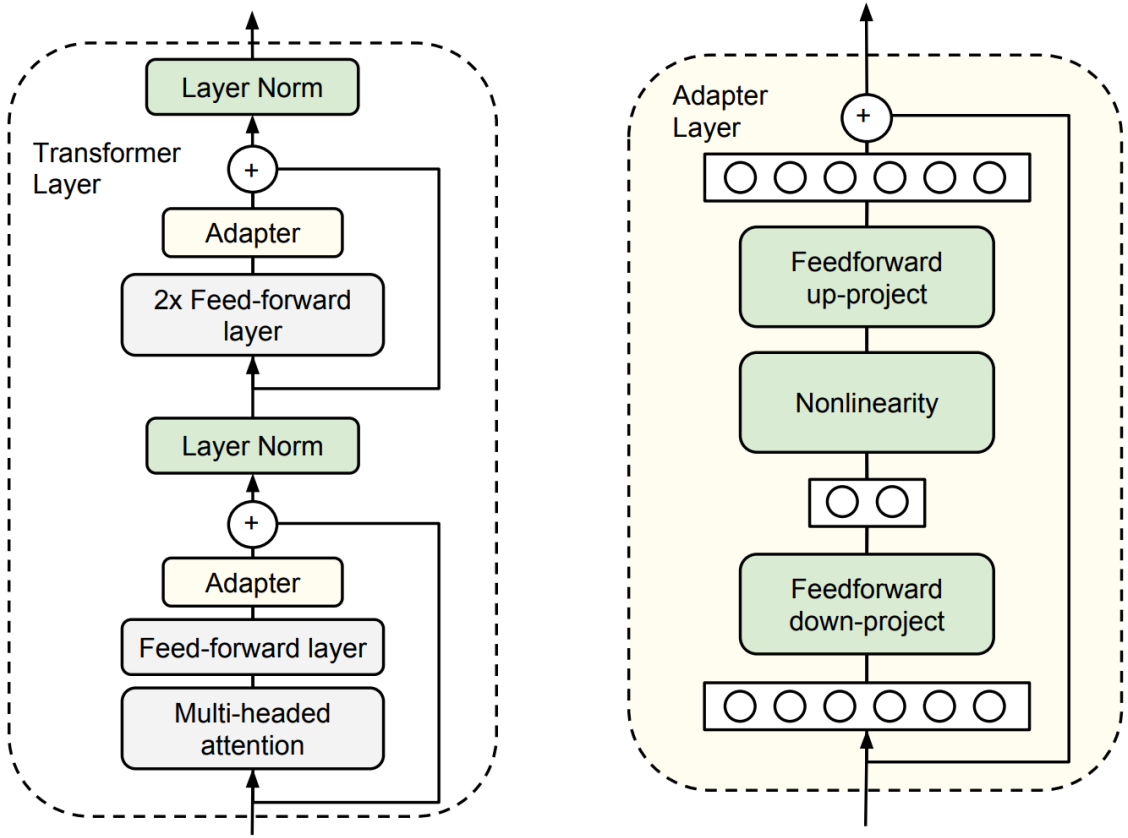
- 目前超过数十亿以上参数的具有强能力的大模型 (例如 GPT-3) 通常在为了适应其下游任务的微调中会呈现出巨大开销。

# 高效微调方案

- 针对全量微调的昂贵问题，目前主要有两种解决方案：
  - Adapt Tuning。即在模型中添加 Adapter 层，在微调时冻结原参数，仅更新 Adapter 层。
  - Prefix Tuning。该种方法固定预训练 LM，为 LM 添加可训练，任务特定的前缀，这样就可以为不同任务保存不同的前缀，微调成本也小。
    - 缺陷：模型可用序列长度减少。由于加入了 virtual tokens，占用了可用序列长度，因此越高的微调质量，模型可用序列长度就越低。

# 高效微调方案

- Adapt Tuning。即在模型中添加 Adapter 层，在微调时冻结原参数，仅更新 Adapter 层。
  - 具体而言，其在预训练模型每层中插入用于下游任务的参数，即 Adapter 模块，在微调时冻结模型主体，仅训练特定于任务的参数。
  - 每个 Adapter 模块由两个前馈子层组成，第一个前馈子层将 Transformer 块的输出作为输入，将原始输入维度 $d$ 投影到 $m$，通过控制 $m$ 的大小来限制 Adapter 模块的参数量，通常情况下 $m << d$。在输出阶段，通过第二个前馈子层还原输入维度，将 $m$ 重新投影到 $d$，作为 Adapter 模块的输出。

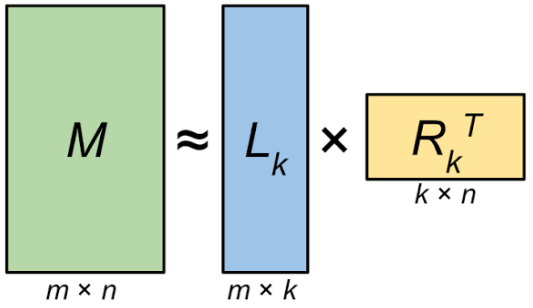


# LoRA

- LoRA: Low-Rank Adaptation of Large Language Models是一种改进的 Adapt Tuning 方法
  - 是微软提出的一项技术，主要用于处理大模型微调的问题。
  - LoRA 建议冻结预训练模型的权重并在每个 Transformer 块中注入可训练层 (秩-分解矩阵)。因为不需要为大多数模型权重计算梯度，所以大大减少了需要训练参数的数量并且降低了 GPU 的内存要求。
  - 研究人员发现，通过聚焦大模型的 Transformer 注意力块，使用 LoRA 进行的微调质量与全模型微调相当，同时速度更快且需要更少的计算。

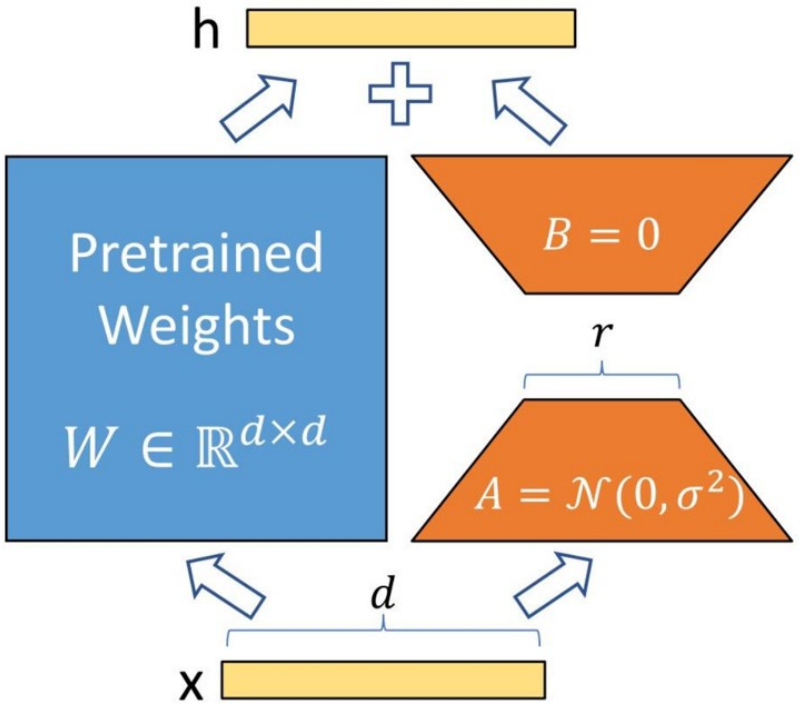
# LoRA

- 微调的本质是实现 $W_0 + \Delta W$
  - 其中$W_0$是预训练学习到的参数
  - $W_0 + \Delta W$是微调之后的参数
- 一些关于$\Delta W$的要求
  - 信息可能是**冗余**的。我们用跟预训练模型一样的参数，来学一个微调的、特定领域的任务
  - 如果用跟预训练一样的算力去学习$\Delta W$的话，是有点浪费的
  - 微调的时候，我们希望原来的模型不要改动太多（可能导致模型能力的遗忘）
- $\Delta W$应该有的性质：
  - 应该是一个低秩(low rank)矩阵
    - 低秩矩阵（Low-Rank Matrix）是指其秩（Rank）远小于其行数和列数的矩阵。其本质特点是信息存在大量冗余，即矩阵中的许多行或列可以由少量特征向量的线性组合表示。低秩矩阵常用于数据压缩、去噪和降维。



# LoRA

- **LoRA示意图**

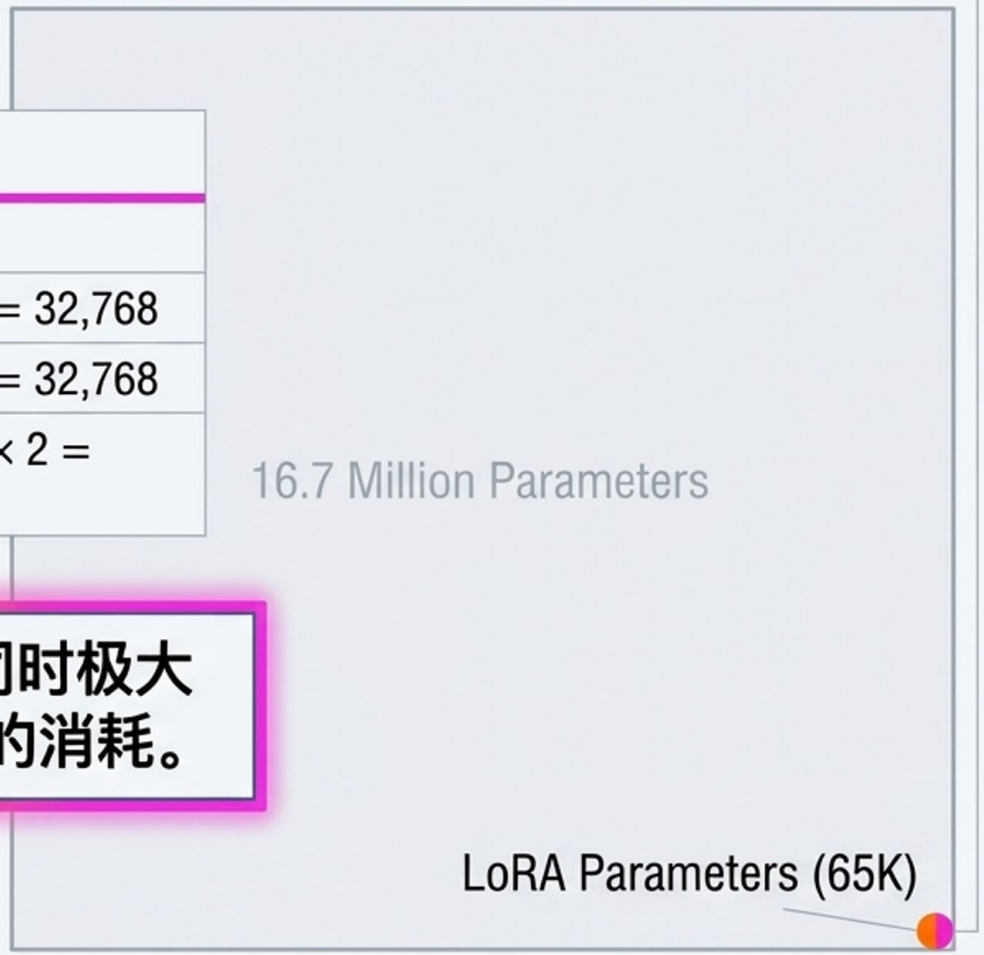


# LoRA

假设某一网络层的维度为 d=4096, k=4096。

| 全量微调 Full FT | LoRA微调 |
| --- | --- |
| 更新整个 $W_{_0}$ 矩阵。 | 假设设置极小的秩 r=8。 |
| 参数量：4096 × 4096 ≈ 1677 万（16.7M）个参数。 | 矩阵 A 尺寸：8 × 4096 = 32,768 |
| | 矩阵 B 尺寸：4096 × 8 = 32,768 |
| | 总训练参数量：32,768 × 2 = 65,536（65K）个参数。 |

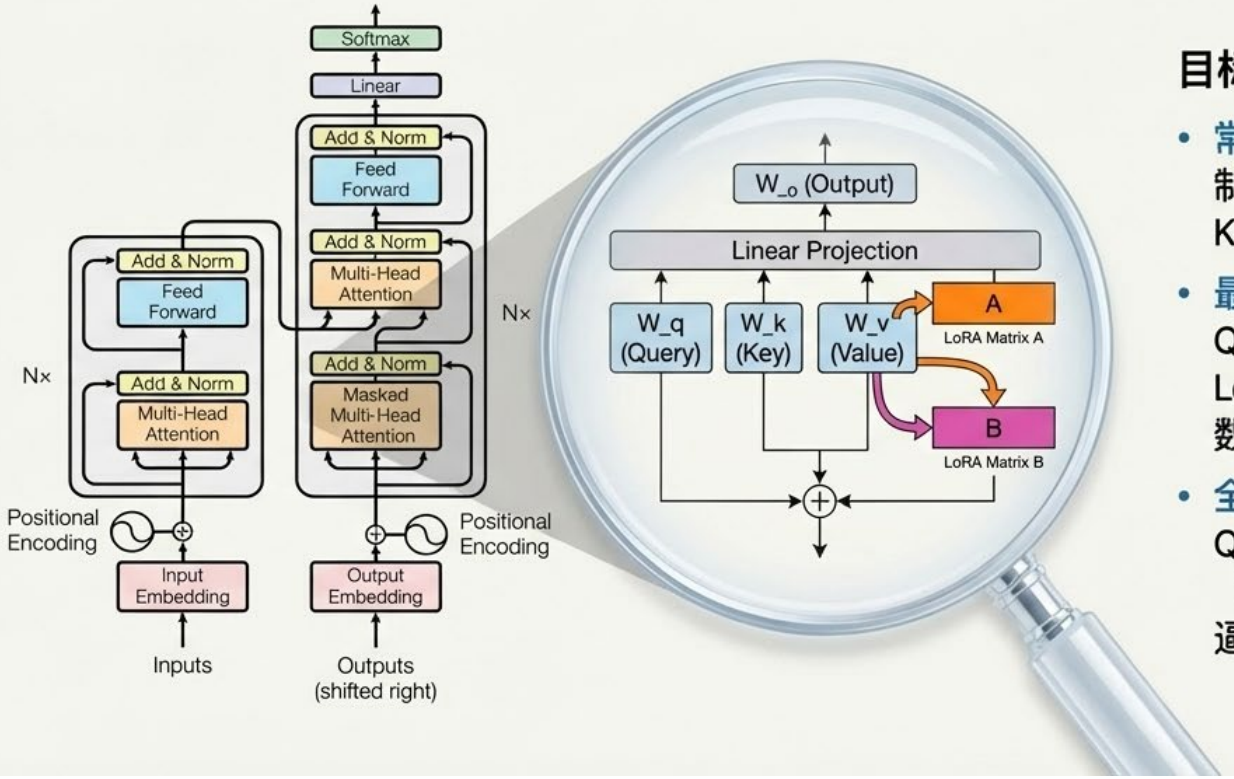
**参数量骤降至原先的 ~0.4%，同时极大降低了对显存（优化器状态）的消耗。**



# LoRA

## Transformer 注入点：目标模块选择

在拥有数以百计线性层的 LLM 中，LoRA 应该放在哪里？




### 目标模块选择策略

- **常规做法**：通常只应用于注意力机制中的线性投影层（即 Query，Key，Value，Output 权重）。
- **最佳实践**：多项实验表明，仅对 Query（$W_q$）和 Value（$W_v$）注入 LoRA 通常能获得最好的性能与参数效率平衡。
- **全连接层注入**：在资源允许时（如 QLoRA 范式），也可对前馈网络（FFN）的所有线性层应用 LoRA 以逼近全量微调效果。

NotebookLM

# LoRA使用方法

- 目前一般通过 peft 库来实现模型的 LoRA 微调。peft 库是 huggingface 开发的第三方库，其中封装了包括 LoRA、Adapt Tuning、P-tuning 等多种高效微调方法，可以基于此便捷地实现模型的 LoRA 微调。
- **PEFT**
  - PEFT (Parameter-Efficient Fine-Tuning) is a library for efficiently adapting large pretrained models to various downstream applications without fine-tuning all of a model's parameters because it is prohibitively costly.
  - https://github.com/huggingface/peft

# LoRA使用方法

- **PEFT实现流程**
  - 确定要使用 LoRA 的层。
  - 对每一个要使用 LoRA 的层，替换为 LoRA 层。
    - 所谓 LoRA 层，实则是在该层原结果基础上增加了一个旁路，通过低秩分解（即矩阵 A 和矩阵 B）来模拟参数更新。
  - 冻结原参数，进行微调，更新 LoRA 层参数。

# LoRA使用方法

```
from transformers import AutoModelForCausalLM
from peft import LoraConfig, TaskType, get_peft_model

device = torch.accelerator.current_accelerator().type if hasattr(torch, "accelerator") else "cuda"
model_id = "Qwen/Qwen2.5-3B-Instruct"
model = AutoModelForCausalLM.from_pretrained(model_id, device_map=device)
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    task_type=TaskType.CAUSAL_LM,
    # target_modules=["q_proj", "v_proj", ...]  # optionally indicate target modules
)
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()
# prints: trainable params: 3,686,400 || all params: 3,089,625,088 || trainable%: 0.1193

# now perform training on your dataset, e.g. using transformers Trainer, then save the model
model.save_pretrained("qwen2.5-3b-lora")
```

# LoRA使用方法

To load a PEFT model for inference:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

device = torch.accelerator.current_accelerator().type if hasattr(torch, "accelerator") else "cuda"
model_id = "Qwen/Qwen2.5-3B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map=device)
model = PeftModel.from_pretrained(model, "qwen2.5-3b-lora")

inputs = tokenizer("Preheat the oven to 350 degrees and place the cookie dough", return_tensors="pt")
outputs = model.generate(**inputs.to(device), max_new_tokens=50)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

# Mixture-of-Experts

- 什么是混合专家模型？
  - 模型规模是提升模型性能的关键因素之一。在有限的计算资源预算下，用更少的训练步数训练一个更大的模型，往往比用更多的步数训练一个较小的模型效果更佳。
  - 混合专家模型 (MoE) 的一个显著优势是它们能够在远少于稠密模型所需的计算资源下进行有效的预训练。这意味着在相同的计算预算条件下，你可以显著扩大模型或数据集的规模。特别是在预训练阶段，与稠密模型相比，混合专家模型通常能够更快地达到相同的质量水平。

# Mixture-of-Experts

- 混合专家模型主要由两个关键部分组成:
  - **稀疏 MoE 层**: 这些层代替了传统 Transformer 模型中的前馈网络 (FFN) 层。MoE 层包含若干“专家”(例如 8 个)，每个专家本身是一个独立的神经网络。
    - 在实际应用中，这些专家通常是前馈网络 (FFN)，但它们也可以是更复杂的网络结构，甚至可以是 MoE 层本身，从而形成层级式的 MoE 结构。
  - **门控网络或路由**: 这个部分用于决定哪些令牌 (token) 被发送到哪个专家。

# Mixture-of-Experts

“More”这个token可能被发送到第二个专家，而“Parameters”这个token被发送到第一个专家。有时，一个token甚至可以被发送到多个专家。 toke的路由方式是 MoE 使用中的一个关键点，因为路由器由学习的参数组成，并且与网络的其他部分一同进行预训练。

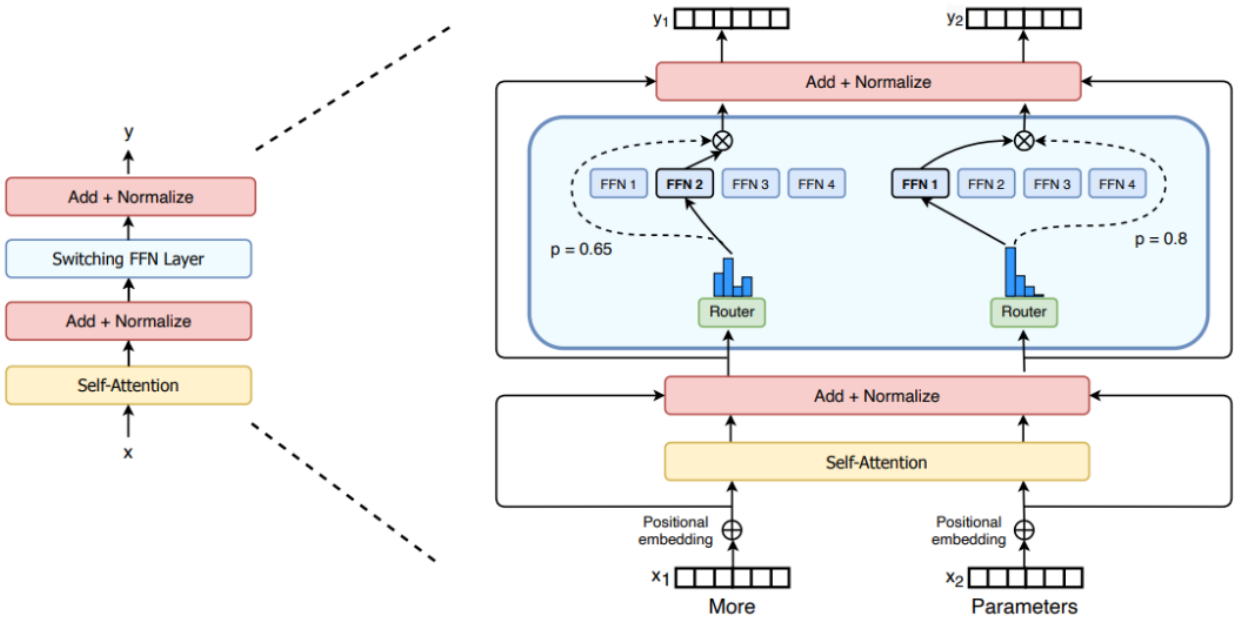



Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens ($x_1$ = “More” and $x_2$ = “Parameters” below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

[Switch Transformers paper](https://arxiv.org/abs/2101.03961) 论文中的 MoE layer

# Mixture-of-Experts

- 混合专家模型 (MoE) 的引入使得训练具有数千亿甚至万亿参数的模型成为可能
  - 通过一个可学习的门控网络 (G) 决定将输入的哪一部分发送给哪些专家 (E)

$$y = \sum_{i=1}^{n} G(x)_i E_i(x)$$

  - 稀疏性
    - 一般只选择G(x)最大的K个专家进行激活
    - 稀疏性的概念采用了条件计算的思想。在传统的稠密模型中，所有的参数都会对所有输入数据进行处理。相比之下，稀疏性允许我们仅针对整个系统的某些特定部分执行计算。这意味着并非所有参数都会在处理每个输入时被激活或使用，而是根据输入的特定特征或需求，只有部分参数集合被调用和运行。
      - 条件计算的概念 (即仅在每个样本的基础上激活网络的不同部分) 使得在不增加额外计算负担的情况下扩展模型规模成为可能。这一策略在每个 MoE 层中实现了数以千计甚至更多的专家的有效利用。